Edge Cases Design

Final submission, updated for the final version: un-planned events the display and the app are likely to meet, and how each one is handled

Shada Nujeidat, Yuval Carmelli, Liat Rokach · 19 October 2026

An edge case is an event we did not plan as the normal flow but that is likely to happen: WiFi drops, a service is down, the user taps the wrong thing, a command arrives at an awkward moment, the power goes out. For each one we define a **user indication** (how the user knows there is a problem) and a **recovery behavior** (how the system keeps working). The last column shows where it is handled in the code (firmware files are under `main/src/`; app files are in the phone app repository).

Handled	implemented and tested on the board
Partly	works, but the indication or the recovery can be better; the planned change is in the last column
Planned	not handled yet; fix planned before the final submission
Accepted	left as is on purpose (rare, low impact), documented

1. Edge cases and responses

A. Network and WiFi

#	Edge case	User indication	Recovery behavior	Status	Where
1	**First start, no WiFi set up**	Header shows "⚠ No WiFi". Clock: "Waiting for WiFi". Weather card: "No WiFi" with "WiFi setup: Settings > Device"	The display starts right away and works offline. A setup portal (WiFi network `SmartDisplay-Setup`) runs in the background; the user joins it from a phone and picks a network. No re-flashing needed	**Handled**	wifi_setup.h
2	**WiFi drops while running**	Header turns to "⚠ No WiFi" within 2 s on every screen. Weather "Updated 10:30" turns red (data is old)	Last weather, events and lists stay on screen. The saved network is retried every 15 s. Everything that does not need internet keeps working	**Handled**	connection_status.h, wifi_setup.h
3	**Saved network gone for good** (moved to another place, router changed)	"⚠ No WiFi"	After 1 min offline the setup portal opens again, so a new network can be chosen. While it is open the saved network is retried every 60 s instead of 15 s so the portal stays responsive. It closes by itself when connected	**Handled**	wifi_setup.h
4	**Wrong WiFi password typed in the portal**	Header stays "⚠ No WiFi"	The setup network stays available, so the user joins it again and re-enters the password	**Handled**	wifi_setup.h
5	**WiFi connected but no internet** (captive portal, e.g. a login page, or the router lost its internet)	Red "⚠ No internet" at the top of every screen (on Home in the greeting row) once no server has answered for 20 s; weather and calendar messages say "No internet"	Every request reports whether a server answered. Weather: one immediate retry, then the regular schedule; calendar every 60 s; Firebase every 2 s. Clears by itself when a server answers again	**Handled**	net.h, connection_status.h

#	Edge case	User indication	Recovery behavior	Status	Where
6	**WiFi drops in the middle of a request**	None: the screen keeps responding	Weather and calendar downloads run in a background task on core 0 (like Firebase), so `loop()` never waits for the network. TLS handshake limit 8 s (15 s for Firebase) instead of the default 120 s	**Handled**	net.h, weather.h, calendar.h
7	**Network blocks time servers** (Technion)	Clock shows "--:--" and "Getting the time..."	ntp.technion.ac.il is asked first, pool.ntp.org and time.google.com are backups	**Handled**	time_date.h

B. Online services (weather, calendar, Firebase)

#	Edge case	User indication	Recovery behavior	Status	Where
8	**Weather service down, error reply, or broken JSON**	Before the first success: "Loading..." on the card and screen. After: the old data with its "Updated" time	Nothing is overwritten by a bad reply. Retried as above. One request brings all 5 cities, so switching city never needs the network	**Handled**	weather.h
9	**Calendar not configured** (no ID / API key)	Calendar: "Calendar not set up" with a hint to add them; home card: "Not set up"	Everything else works normally	**Handled**	calendar.h, calendar_screen.h
10	**Wrong calendar ID or API key**	“Couldn’t load events” with a specific hint: “Check the calendar ID (Setup page) and that the calendar is public” (404) or “Check the calendar API key (Settings > Device > Setup page)” (400/403)	Retried every minute; events from the last successful check stay on screen	**Handled**	calendar_screen.h
11	**No upcoming events**	"No upcoming events / Your calendar is clear"	-	**Handled**	calendar_screen.h
12	**Event title in Hebrew, very long, or with emoji**	Hebrew shown right-to-left; long titles cut with "…" on one line; characters the fonts lack shown as "?" (never as empty boxes)	-	**Handled**	shared_ui.h
13	**Firebase sign-in fails** (wrong login, sign-in method off) or voice commands not set up	Settings → Device: Voice commands "Sign-in failed" / "Database error" / "Not set up"; an empty list says "Voice commands are off: see Settings > Device". In the app: an answer at once (the display no longer reports in, so the server treats it as offline)	Sign-in retried every 30 s; the rest of the display works. The login is fixed from the phone on the Setup page (no code)	**Handled**	firebase_sync.h, settings.h, config.h
14	**Firebase login expires** (tokens last 1 hour)	None	Renewed 5 minutes before it expires; if a request still gets "expired", it signs in again on the next round	**Handled**	firebase_sync.h
15	**Firebase unreachable**	App: answered by the server from the `/lists` copy	Retried every 2 s, error logged once. Commands wait in the /commands queue; list changes made meanwhile are kept and copied to /lists when the connection is back	**Handled**	firebase_sync.h

C. Voice commands (phone app → Firebase → display)

#	Edge case	User indication	Recovery behavior	Status	Where
16	**Phone is offline**	App: "You're offline. Connect to the internet to send voice commands." Recording is refused	Banner disappears when the phone is back online	**Handled**	app: pwa.js, app.js
17	**Microphone permission denied**, or app opened without HTTPS	"Could not open your microphone. Check browser permissions." / "Open this app over HTTPS in Safari or Chrome…"	Nothing is sent; the user fixes the permission and tries again	**Handled**	app.js
18	**Very long recording**	Recording stops	Stops automatically after 30 s and is sent	**Handled**	app.js
19	**Gemini's reply is not usable, or what was said is not a list command**	The specific reason from rows 38-49 (e.g. “Say “add” or “remove”, then one item…”). Only a broken Gemini reply (not valid JSON) gives the general “Could not understand a valid command. Please try again.”	The server checks the JSON (action, list, one non-empty item that was really said) and answers 422; nothing is written to Firebase, the display is not touched	**Handled**	server.js, commands.js
20	**Gemini busy** (503)	"Understanding your request…" takes longer	The server retries 3 times, 2 s apart, then reports an error	**Handled**	server.js
21	**Gemini doesn't answer at all**	"The voice service is busy right now. Please try again in a moment." after at most 20 s	Each Gemini call is cancelled after 20 s, so the app never waits forever	**Handled**	server.js
22	**Many requests from one address** (a stuck phone, or someone abusing the open API)	"Too many requests. Wait a minute and try again."	At most 15 commands and 120 status checks per minute per address, so the Gemini key and the database are protected	**Handled**	server.js, commands.js
23	**Wrong item added or removed by voice**	An “Undo (remove milk)” button under the result, only when the list really changed (not for “already on the list”)	Sends the opposite command without a recording; the same checks and queue as a voice command	**Handled**	app.js, server.js
24	**Display off or offline when a command is sent**	App answers at once, the same as when online (e.g. "Added milk to your shopping list"); the user doesn't need to know the display is offline	The display writes `/device/lastSeen` every 10 s; after 25 s without it the server answers from `/lists` plus the commands still waiting. The command stays in the `/commands` queue and is applied when the display is back	**Handled**	server.js, commands.js, firebase_sync.h
25	**Same item changed on the screen while offline** (e.g. "add milk" by voice is queued, then milk is ticked off on the screen)	The later change is what the user sees. If a queued command is older: a log line "… was changed on the display after this request"	Every item remembers when it was last added or removed; a command older than that is ignored (the later change wins). Removed items are remembered for a while for this. Checked on a computer with the real lists.h: all 573 possible sequences of up to 6 changes, no conflicts	**Handled**	lists.h, tools/test_lists
26	**Offline answer differs from the final result** (the item was also changed on the screen, which the server can't see yet)	The app said "Added milk", but a later tap removed it	The display's list is always right (later change wins). Rare: needs a voice command and a tap on the same item while the display is offline	**Accepted**	server.js
27	**Display restarts with commands waiting**, or before the clock is synced	None	Commands are only removed from the queue after they were applied and saved, and applying one twice changes nothing. A tap made before the clock synced is timed from boot and corrected when it syncs	**Handled**	firebase_sync.h, lists.h

#	Edge case	User indication	Recovery behavior	Status	Where
28	**Item already on the list** ("add milk" twice)	App: “milk is already on your shopping list.” (the server checks the display’s list and sends nothing). If the display’s list changed meanwhile, the display’s pop-up says the same	No duplicate. Matching ignores upper/lower case	**Handled**	lists.h
29	**Remove an item that is not on the list**	App: “eggs isn’t on your shopping list.” (checked by the server before sending). If the list changed on the display in the meantime, the display’s red pop-up “eggs isn’t on your shopping list” and the same text in the app	List unchanged; nothing is sent to the display when the server already knows the item is missing	**Handled**	lists.h, server.js, commands.js
30	**List full** (20 items)	Red pop-up “Your shopping list is full” on the display, and the same text in the app	Item not added; the user ticks items off to make room	**Handled**	lists.h
31	**Item name too long**	Shown on one line with "…"	Stored up to 47 bytes, cut at a letter boundary so Hebrew letters are never broken	**Handled**	lists.h
32	**Empty item, unknown list or action** (bad JSON written to /commands)	Red pop-up "The display couldn't understand this command"; app shows failure	Lists unchanged	**Handled**	firebase_sync.h
33	**Several commands in a row**	Each opens its list with its pop-up	Commands are applied oldest first; screen updates wait in a queue (4 places). If the screen falls behind, extra pop-ups are dropped but the lists are still correct	**Handled**	firebase_sync.h
34	**Two phones send at the same moment**	Each app gets the reply to its own command	Every command gets its own place in the `/commands` queue (Firebase push id), so nothing is overwritten	**Handled**	server.js
35	**/commands or /status deleted in the console**	A deleted command is not applied; the app says “Sent, but the display hasn’t confirmed yet”	Replies are matched by command id, so a deleted /status or /command doesn’t confuse later commands; unread replies in /status are removed after 10 min	**Handled**	firebase_sync.h, server.js
36	**Command arrives while the user is using another screen** (e.g. Settings)	If the screen was touched in the last 10 s: only the pop-up; otherwise the list it changed opens with the pop-up	The user is never pulled away from what they are doing. An untouched list goes back home after 30 s	**Handled**	display_ui.h
37	**Voice “add milk” while “milk” is ticked (3 s undo window)**	App: “milk is already on your shopping list.”; milk is still removed when the 3 s end	The server sees milk still on the list, so it doesn’t send the command. Rare (needs the command within the same 3 seconds); saying it again adds milk	**Accepted**	server.js, lists_screen.h

C2. What the user says (understanding the voice command)

Gemini turns the recording into {transcript, action, item, list}. The server then checks the transcript itself by fixed rules, so a wrong guess by the model never reaches the list. All cases below are automated tests (voice.test.js in the app). They were also tested by voice with the real app and Gemini on 19 October 2026: 35 sentences in English and Hebrew (valid commands, every rejection type, display off, rate limit), all with the expected result.

#	Edge case	User indication	Recovery behavior	Status	Where
38	**No add/remove verb** ("milk to my shopping list", "I need milk", "milk")	"Say “add” or “remove”, then one item, e.g. “Add milk to my shopping list”."	Rejected: the server checks the words Gemini heard for an add or remove verb (English and Hebrew), even if Gemini guessed "add"	**Handled**	commands.js
39	**A question or small talk** ("what time is it", "hello")	Same message	Rejected; never turned into a list item	**Handled**	server.js, commands.js
40	**Silence, noise or a cut-off recording**	“I didn’t hear anything. Hold the button while you speak.” / “Could not understand a valid command. Please try again.”	Nothing is sent to the display	**Handled**	commands.js
41	**Several items** ("add milk and eggs", "add milk, eggs")	"One item at a time, please"	Rejected. One item that contains "and" ("mac and cheese") is still accepted	**Handled**	server.js, commands.js
42	**Add and remove in one sentence** ("add milk and remove eggs")	“Say “add” or “remove”, then one item, e.g. “Add milk to my shopping list”.” (the command help)	Rejected	**Handled**	commands.js
43	**Negation** ("don't add milk", "אל תוסיף חלב")	"Nothing was changed."	Rejected	**Handled**	commands.js
44	**A list that doesn't exist** ("add milk to my work list", "לרשימת העבודה")	"There are two lists: shopping and to-do."	Rejected instead of guessing a list	**Handled**	commands.js
45	**No list named** ("add bread", "add cooking")	"Say which list: “to my shopping list” or “to my to-do list”."	Rejected: the list must be named, no guessing ("cooking" could be either). A named list always wins over Gemini's choice	**Handled**	commands.js
46	**Gemini invents or mishears the item** (item not in what was said)	"I couldn't make out the item."	Every word of the item must appear in the transcript	**Handled**	commands.js
47	**Gemini answers the opposite action** (said "remove", Gemini returned "add")	Command help message	The verb in the sentence must match the action	**Handled**	commands.js
48	**Strange but explicit item** ("add where are you to my shopping list")	Added as "where are you"	Decided by the sentence structure, not by what the item means	**Handled**	server.js, commands.js
49	**Item too long (over 40 characters, about 23 in Hebrew) or only filler words (“add the”)**	"That item is too long" / "I couldn't make out the item"	Rejected; the display keeps up to 47 bytes per item	**Handled**	commands.js
50	**Fillers and capitals** ("please add the Milk.")	Added as "Milk"	"please", "a", "the", "some" at the start are dropped; punctuation ignored	**Handled**	commands.js

D. Touch screen and user interface

#	Edge case	User indication	Recovery behavior	Status	Where
51	**Touch while the screen is dimmed or off**	Screen wakes to the saved brightness	The waking touch does not press anything under the finger	**Handled**	idle.h
52	**Tap registered twice, or a scroll taken as a tap** (resistive touch)	None	LVGL acts on one complete press-and-release and ignores a press that turns into a scroll; buttons are at least 44 px	**Handled**	display_ui.h

#	Edge case	User indication	Recovery behavior	Status	Where
53	**All Home widgets hidden** (Settings → Home screen)	"All widgets are hidden - Turn them on in Settings > Home screen"; the greeting row and ⚙ stay	The gear always stays, so the user can turn widgets back on	**Handled**	home.h
54	**A widget gets very little room** (one widget Big, all four shown)	Compact versions: one or two lines (e.g. "Shopping 7 / To-do 2", "Today 18:00 / Team meeting")	Every widget is at least 48 px tall and has a compact version for it; all 80 layouts checked on a computer	**Handled**	home_layout.h, tools/test_home_layout
55	**Lists widget hidden, a voice command arrives**	The list opens with the pop-up as usual	Hiding a widget only removes it from Home; lists and voice commands keep working. Hidden Weather / Next event stop downloading	**Handled**	display_ui.h, weather.h, calendar.h
56	**Item ticked by mistake**	Row crossed out	Removed only after 3 s (a pop-up says how to keep it); tapping it again keeps it	**Handled**	lists_screen.h
57	**Brightness slider pulled to the bottom**	Screen stays dimly lit	Minimum is 10/255 (4 %), so the screen can never be turned fully dark by mistake	**Handled**	settings.h
58	**Empty list**	"Shopping list is empty" / "Nothing to do"	-	**Handled**	lists_screen.h
59	**Text longer than its space** (city, item, event)	One line with "…"; Hebrew right-to-left	-	**Handled**	shared_ui.h
60	**Board not touched for a long time**	Dims or turns off after the chosen time (1-30 min) if power saving is on	Any touch wakes it	**Handled**	idle.h

E. Data, storage and power

#	Edge case	User indication	Recovery behavior	Status	Where
61	**Power cut or reset**	Normal start; clock “Getting the time...” (or “Waiting for WiFi” without WiFi) until synced	Settings, both lists and the last weather are saved in flash and restored; the last events too, shown once the clock is set. Time is synced again from NTP (the board has no clock battery)	**Handled**	storage.h, lists.h
62	**List changed without WiFi** (instructor feedback)	None needed: the change shows at once	Saved in flash with the time of the change; copied to Firebase `/lists` as soon as WiFi is back. Voice commands sent meanwhile are merged by time (row C)	**Handled**	lists.h, firebase_sync.h
63	**Saved data from an older firmware**, or out-of-range saved settings	Lists from the previous format are converted (items kept); a setting outside its range shows its default	Saved lists are checked by size and format version: version 1 (names only) is converted once at boot, a damaged list starts empty without touching the other; every saved setting is limited to its allowed range	**Handled**	lists.h, settings.h, idle.h
64	**Flash write fails**	Only in the log: "[Lists] Could not save to flash, retrying"	The list stays correct in memory and the write is retried every 10 s until it works (checked on a computer with a simulated broken flash)	**Handled**	lists.h
65	**Clock not synced yet**	"--:--" and "Waiting for WiFi" / "Getting the time..."; world clock without hands	Calendar waits for the time before loading, so "Today" is never wrong	**Handled**	time_date.h, calendar.h
66	**Summer / winter time change**	None	Israel's rule and each world-clock city's own rule are built in; the change happens automatically	**Handled**	time_date.h

#	Edge case	User indication	Recovery behavior	Status	Where
67	**Firmware freezes** (a bug, or a request that never returns)	Board restarts by itself; Settings → Device → Last restart: "Watchdog (froze)"	A 60 s watchdog on `loop()`; all state comes back from flash. A health line every 10 min logs free and lowest memory for long tests	**Handled**	health.h
68	**Someone nearby tries to join the setup network**	None	The setup network only opens when the display has no WiFi, and it has a password (smartdisplay, shown in Settings → Device while offline); passwords and keys are never shown on the Setup page; the firmware-upload page is turned off	**Handled**	wifi_setup.h
69	**Memory runs low or a task runs out of stack**	Board restarts	Fixed-size buffers for lists and events; weather and calendar never run at the same time; free memory printed at each sign-in; after a restart all state comes back from flash	**Handled**	firebase_sync.h
70	**Weak USB power** (laptop port, long cable)	Board restarts when WiFi and backlight peak	All state is restored after the restart. Use a 5 V / 1 A adapter for the display	**Accepted**	README
71	**“Clear list” tapped by mistake**	The button turns red: “Tap again to clear all 6”	Nothing is removed without a second tap within 4 s; otherwise the button goes back to “Clear list”. A confirmed clear is one change, saved in flash and copied to Firebase; a voice “add” sent before the clear is ignored, one sent after it is applied (checked on a computer with the real lists.h)	**Handled**	lists_screen.h, lists.h, tools/test_lists

2. Robustness self-check (6 criteria from the course guide)

Criterion	What the project has	Gap / next step
1. Feature-rich	LVGL touch UI with 6 screens, voice commands through Gemini and Firebase, Google Calendar, weather for 5 cities in one request, world clock with analog dials, Hebrew right-to-left text, power saving. Firebase runs in its own FreeRTOS task on the second core	Done: weather and calendar download in a background task
2. Generic / modular	One folder per feature with the same pattern (`init<Feature>()`, `handle<Feature>()`, `build<Name>Screen()`); data modules contain no drawing code; all colors and fonts come from one design system (`theme.h`); hard-coded values listed in `Documentation/parameters.md`	Optional "demo data" mode for showing the UI without internet
3. Configurable	WiFi through the setup portal (no re-flashing); city, 24-hour time, brightness, dark/light mode and power saving from Settings, all saved; secrets in `secrets.h` with a template	Done: Firebase login and calendar on the Setup page (no code). Still fixed: touch calibration (`test_touch` prints the raw values to re-calibrate)
4. Stable	Non-blocking WiFi, timeouts on every request, retries, offline mode with data kept in flash, lists protected by a lock, LVGL used from one task only, unit tests for every part	Done: Settings → Device (internet, signal, voice state, uptime, last restart, memory), watchdog, flash retry, certificate check. Next: a 24-72 h soak test using the health log
5. Organized	Code split by feature with comments; only `handleTouch()` changes the open screen; each hardware part is accessed by one module; single board, no external wiring	Enclosure (see Enclosure Sketch)
6. User-friendly	Every state is shown on screen (No WiFi, Loading, Not set up, Updated time, red/blue pop-ups); the app shows the result of each command; wrong input is rejected with a clear message; scrollable lists and a status bar on every screen	Tested with family and friends; changes made from their feedback (see Feasibility Tests 3.4). Next: another short test of the final version

Totals: 71 edge cases. Detailed messages and log lines for each state: Documentation/errors_and_states.md in the firmware repository.